Implement the complete authentication and authorization functionality now. Keep…

Worked for 11m 35s

Implemented and committed complete authentication and authorization. Key functionality:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me
- bcrypt password hashing with cost 12
- Normalized case-insensitive email registration
- Server-assigned user role regardless of submitted privilege fields
- Expiring HS256 JWTs with issuer, audience, signature, and subject validation
- Startup rejection when JWT_SECRET is missing or too short
- Database-backed active status and role checks on every authenticated request
- Reusable authenticate and requireRole('admin') middleware
- Centralized safe API errors
- TEST_DATABASE_URL support
- Updated environment, API, database, and AI usage documentation Verification:
- npm test: 34 tests passed across 7 files
- Live PostgreSQL authentication integration tests ran successfully
- npm run build: passed
- git diff --check: passed
- npm audit: 0 vulnerabilities
- Test account cleanup verified: 0 records remained
- .env remains ignored and untracked
- No existing migration needed modification Commit: a23848db11637b95b2f64447e4d8d1405de3eed6 feat: implement authentication and authorization Exact final Git status:

main...origin/main [ahead 1]

Nothing was pushed. Skipped: inventory APIs and frontend, as requested. Risk: the current ignored .env database password did not authenticate, so update its DATABASE_URL and add a private JWT_SECRET before running the server normally.

Edited 13 files+546-4View changesUndoView changes

test/integration/auth.integration.test.tstest/integration/auth.integration.test.ts+194-0 src/app.tssrc/app.ts+7-0 src/auth-routes.tssrc/auth-routes.ts+31-0 Show 10 more files

11:40 AM

Implement the complete vehicle inventory API now. I want this properly implemen…

Worked for 13m 28s

Implemented and committed the complete vehicle inventory API. Key functionality:

- Public vehicle listing, searching, filtering, sorting, and pagination
- Public active vehicle detail endpoint
- Admin-only vehicle creation, updates, soft deletion, and restocking
- Strict field, numeric, UUID, pagination, filter, and sort validation
- Parameterized PostgreSQL queries
- Opening-stock and restock audit receipts
- Atomic concurrent restocks

- Idempotent restocks using request UUIDs
- Database-backed authentication and admin authorization
- Soft-deleted vehicles excluded from public reads
- No sensitive or internal fields in public responses Migration 002_create_stock_receipts.sql was applied successfully to the existing development database. A second run reported no pending migrations. A separate dealership_inventory_test database was created for isolated integration testing. Checks completed:
- npm test: 57 tests passed across 8 files
- Live PostgreSQL integration tests passed
- Authentication regression tests passed
- npm run build: passed
- git diff --check: passed
- Test records remaining: zero
- No dependencies or tracked secrets added Commit: 6eaf193fd3a9dd3c9e588cc660819e6d0db6642a feat: implement vehicle inventory APIs Exact Git status:

main...origin/main [ahead 1]

Nothing was pushed. Skipped: purchases, cart, favourites, notifications, category management, and frontend. Risk: npm run migrate encountered one transient Windows/Node tsx memory error; the compiled migration runner completed successfully and verified idempotency.

Edited 10 files+564-11View changesUndoView changes

test/integration/vehicles.integration.test.tstest/integration/vehicles.integration.test.ts+200-0 migrations/002_create_stock_receipts.sqlmigrations/002_create_stock_receipts.sql+14-0 src/app.tssrc/app.ts+2-0 Show 7 more files

12:13 PM

I need to correct how the existing implementation was developed before we add a...

Worked for 39m 32s

Completed the retrospective TDD remediation without rewriting published history or pushing. The original Kata explicitly requires protected vehicle endpoints, visible Red-Green-Refactor history, AI co-authorship, and raw AI conversations or public links. C:\PROJECTS\Dealership Inventory\reference-docs\AI_Kata_Car_Dealership_Inventory_System_V2.pdf Baseline audit The existing 57 tests covered most core behavior, but the audit found three real defects:

- Integration tests could silently fall back to the development database.
- Bcrypt accepted passwords that differed only after its 72-byte truncation boundary.
- Vehicle reads were public despite the Kata marking them protected.
- Restock retries returned current stock instead of the original operation result. TDD cycles Cycle Red commit and observed failure Green commit Test database isolation b648c70, 063f068 — missing guard caused focused suites to fail cb493e3 — requires TEST_DATABASE_URL ending in _test Bcrypt boundary cec6145 — overlong altered password returned 200 instead of 401 a56f91e — rejects login passwords over 72 bytes Protected vehicle reads d7d0c0b — unauthenticated vehicle listing returned 200 instead of 401 a24ba12 — applies existing authentication middleware Restock replay result 467218a — replay returned quantity 4 instead of original 3 2dd1e92 — migration 003 stores and returns the receipt's resulting quantity

No behavior-preserving refactor was justified, so no artificial refactor commits were created. Documentation commit:

- 06803de — clarified isolated test setup, honest historical TDD status, Codex usage, and missing raw AI records. Verification
- Full suite: 62 tests passed across 9 files
- Skipped integration tests: 0
- Live database: verified dealership_inventory_test
- TypeScript build: passed

- git diff --check: passed
- Test fixture records remaining: 0|0|0
- Migration 003 applied to development successfully
- Compiled migration runner reported up-to-date twice
- Working tree: clean
- Nothing pushed Exact Git status:

main...origin/main [ahead 10]

PROMPTS.md still needs the original, unedited AI conversations or public chat links. Supplied prompts and generated summaries alone do not meet the Kata requirement. Skipped: live rollback injection, maximum stock overflow, page-two pagination, wrong JWT issuer/audience, and server-port validation remain useful coverage candidates. Risk: npm run migrate still fails before application code inside tsx with Windows/Node uv_os_get_passwd ... ENOMEM; node dist/migrate.js works and verified migration idempotency.

Edited 14 files+142-47View changesUndoView changes

test/integration/auth.integration.test.tstest/integration/auth.integration.test.ts+18-2
test/integration/db.integration.test.tstest/integration/db.integration.test.ts+3-3
test/integration/migrate.integration.test.tstest/integration/migrate.integration.test.ts+5-4 Show 11 more files

2:37 PM

I want to restructure the entire repository properly before continuing with mor...

Worked for 27m 4s

Repository restructuring is complete, committed locally, and not pushed. Completed

- Moved all backend code, migrations, configuration, and tests into [`backend/`](C:/PROJECTS/Dealership Inventory/backend).
- Added a minimal Vite + React + TypeScript + Tailwind scaffold in [`frontend/`](C:/PROJECTS/Dealership Inventory/frontend).
- Added npm workspaces and root commands for backend/frontend development, builds, tests, migrations, and startup.
- Preserved root .env, Docker Compose, documentation, and ignored local credentials.
- Verified frontend /api and /health proxy configuration against Express.
- Updated migration and .env path handling for the workspace layout.
- Replaced authentication and vehicle implementation summaries with exact recovered prompts.
- Added the exact backend foundation, TDD remediation, and current restructuring prompts.
- Updated README commands, layout, migration paths, and AI-record status.
- Kept the warning that raw AI transcripts or public chat links still need to be supplied. The implementation follows the Kata requirements from the supplied PDF. C:\PROJECTS\Dealership Inventory\reference-docs\AI_Kata_Car_Dealership_Inventory_System_V2.pdf Verification
- Baseline: 62 tests passed; backend build passed.
- Final npm test: 62/62 passed across 9 files, including PostgreSQL integration tests using the dedicated _test database.
- npm run build: backend and frontend passed.
- npm run build:frontend: passed.
- npm run migrate twice: both reported the schema up to date.
- Backend /health: returned {"status":"ok"}.
- Frontend development server: returned HTTP 200.
- Frontend /health proxy: returned {"status":"ok"} from Express.
- npm install: completed with zero vulnerabilities.
- git diff --check: passed.
- Relative Markdown links: passed.

- Working tree: clean. The previous tsx migration failure is no longer present in the root command: npm run migrate now uses the verified compiled runner and succeeded twice. Docker Compose configuration validation was not run because docker is not currently available on PATH. No containers, volumes, or database data were removed. Local commits
- e4ffa39 — refactor: separate backend and frontend workspaces
- edc1da0 — docs: update workspace guide and prompt history Both contain the required Codex co-author trailer. Final status:

main...origin/main [ahead 12]

Nothing was pushed. The next feature should be handled as a genuine test-first cycle—most likely the required direct-purchase workflow before building the full frontend inventory experience.

Edited 19 files+276-55View changesUndoView changes

backend/package.jsonbackend/package.json+2-5 backend/src/db.tsbackend/src/db.ts+5-1
frontend/index.htmlfrontend/index.html+12-0 Show 16 more files

3:15 PM

push it on github with codex as coauthor

3:18 PM

Worked for 1m 7s

Pushed successfully to GitHub.

- Branch: main
- Remote: origin
- Latest commit: edc1da0
- Both new commits include Co-authored-by: Codex noreply@openai.com
- Local main is synchronized with origin/main
- Working tree is clean

3:19 PM